Lehrerexemplar · Algo Quest Welt 6 – Der Endgegner · DS 39–40

Erwartungshorizont: Klausur „Der Ladepark“

Erwartungshorizont / Bewertungsbogen für _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten · P1–P10: Verweis auf die „Top 10 Punktverluste“ aus DS 38

Teilaufgabe	I	II	III	BE
Erwartete Prüfungsleistungen	18	32	10	
a) Aufbau der Klasse Ladesaeule erläutern mit den Kernpunkten: Attribute als Eigenschaften der Objekte (nummer, leistung, aktuell als Referenz auf einen Ladevorgang); Konstruktoren (c) zur Erzeugung und Initialisierung von Objekten, überladen: Ladesaeule(nummer) mit Standardleistung 11 kW, Ladesaeule(nummer, leistung) mit übergebener Leistung; Operationen zum Abfragen (getNummer, getLeistung, istFrei), Verändern des Zustands (starten, beenden) und Berechnen (preisProKWh); Sichtbarkeit - (privat, nur innerhalb der Klasse) bzw. + (öffentlich): Datenkapselung, Zugriff nur über Operationen. *Typische Punktverluste: P10*	3	1		
b) Klassendiagramm um Schnellladesaeule ergänzen mit den Kernpunkten: Klasse Schnellladesaeule mit Vererbungspfeil (hohle Dreiecksspitze) zu Ladesaeule; neue Attribute steckertyp: Zeichenkette und zuschlag: Ganzzahl (privat); Konstruktor mit nummer, leistung, steckertyp, zuschlag; preisProKWh(): Ganzzahl wird überschrieben; ggf. getSteckertyp. *Typische Punktverluste: P3, P10*	1	2		
c) besteFreieSaeule implementieren (dynamische Reihung) mit den Kernpunkten: Methodenkopf mit Rückgabetyp Ladesaeule; Durchlauf aller Elemente von saeulen (0 … getLength() − 1 oder for-each); Cast (Ladesaeule); nur freie Säulen (istFrei); Maximum der Leistung mit null-Behandlung, strenges > (kleinster Index bei Gleichstand); Rückgabe null, wenn keine Säule frei ist oder saeulen leer ist, ohne Laufzeitfehler. *Typische Punktverluste: P1, P3, P4, P6*	2	4		
d) wartezeit implementieren (Schlange unverändert) mit den Kernpunkten: Methodenkopf; Hilfsschlange (oder Rotieren mit bekannter Länge); dequeue nur nach isEmpty-Abfrage; Cast (Ladevorgang); Vergleich mit equals; Summe der Dauern nur vor dem Treffer; Rückgabe −1, wenn nicht enthalten; vollständige Wiederherstellung in alter Reihenfolge (auch nach dem Treffer und ohne Treffer); Rückgabe erst nach der Wiederherstellung. *Typische Punktverluste: P1, P2, P4, P5*	2	4	1	
e) Struktogramm spitzenstunde entwickeln (2D-Reihung) mit den Kernpunkten: Kopf mit Signatur spitzenstunde(): Ganzzahl; äußere Zählschleife über die Stunden (Spalten 0 bis 23); innere Zählschleife über alle Säulen (Zeilen), Summe je Stunde neu bei 0 beginnen, energie[s][h]; Vergleich mit dem bisherigen Maximum, Stunde merken, strenges > für die früheste Stunde; gib … zurück; Notation nach EH §5. *Typische Punktverluste: P6, P9*	1	4	1	
f) Ablauf von sortiereProtokoll dokumentieren (Tracetabelle) mit den Kernpunkten: 12 Schleifendurchläufe; Vertauschungen bei pos = 1, 2, 3, 2 (4 Vertauschungen); Sonderfall pos = 0 ohne Vergleich; Endbelegung 42, 42, 25, 18, 9; Ende bei pos = 5. *Typische Punktverluste: P8*	2	3		
g) Sortierverfahren analysieren mit den Kernpunkten: Ein Element wird durch Vertauschen mit dem linken Nachbarn so lange nach links bewegt, wie es größer ist (Einfügen in den sortierten Bereich, vgl. Einfügesortierung); danach läuft pos wieder nach rechts; pos = 0: das Element steht ganz vorn, es gibt keinen linken Nachbarn – ohne die Abfrage würde getItem(−1) einen Laufzeitfehler auslösen; Invariante: protokoll[0 … pos − 1] ist absteigend sortiert; Erhöhung von pos nur, wenn links ≥ rechts; jede Vertauschung beseitigt genau eine Fehlstellung → Abbruch bei pos = Länge, dann ist alles sortiert. *Typische Punktverluste: P1, P10*		2	2	

Teilaufgabe	I	II	III	BE
h) Vertauschungen bestimmen, Verdopplung abschätzen mit den Kernpunkten: absteigend sortiert: 0 Vertauschungen (n − 1 Schleifendurchläufe); streng aufsteigend: Element mit Index k wird k-mal getauscht → 1 + 2 + … + (n − 1) = n(n − 1)/2 Vertauschungen; Verdopplung: (2n)(2n − 1)/2 ≈ 4 · n(n − 1)/2 → Laufzeit etwa vervierfacht (quadratisches Wachstum), z. B. n = 100: 4950 → n = 200: 19 900. *Typische Punktverluste: P10*	1	2	1	
i) Rekursionstabelle zu ueberlast mit den Kernpunkten: Rekursionstiefe 0 bis 4 mit h = Tiefe; puffer bei den Aufrufen 60, 120, 90, 30, 40; Begrenzung auf 120 in Stunde 0; Abbruch in Stunde 4 (40 + 100 − 150 = −10 < 0); Rückgabe 4 in allen Zeilen. *Typische Punktverluste: P7, P8*	2	2		
j) Prinzip der Rekursion erläutern mit den Kernpunkten: Problem wird auf ein kleineres gleichartiges Problem zurückgeführt: Selbstaufruf für die verbleibenden Stunden ab h + 1 mit dem neuen Pufferstand; Abbruchbedingungen (Basisfälle): Prognose zu Ende (h = Länge) → −1, Überlast → h; das Ergebnis des Selbstaufrufs wird unverändert zurückgegeben; Terminierung, da h bei jedem Aufruf um 1 wächst. *Typische Punktverluste: P7, P10*	1	2		
k) Rekursionstiefe begründen mit den Kernpunkten: Die Rekursionstiefe jedes Aufrufs ist gleich seinem Parameter h (Erstaufruf: Tiefe 0, h = 0; jeder Selbstaufruf erhöht beides um 1); −1 wird nur im Basisfall h = Länge zurückgegeben und dann bis zum Erstaufruf weitergereicht → letzter Aufruf hat die Tiefe n; umgekehrt: hat der letzte Aufruf die Tiefe n, so ist h = n, in den Stunden 0 … n − 1 trat keine Überlast auf (sonst Abbruch vorher) und es wird −1 zurückgegeben; bei Überlast in Stunde h < n endet die Rekursion bei Tiefe h < n. *Typische Punktverluste: P7, P10*		1	2	
l) Binären Suchbaum zeichnen mit den Kernpunkten: Wurzel 518; links 274 (133; 402 mit 350, 461); rechts 730 (615 mit rechtem Kind 688; 911).	2			
m) anzahlImBereich analysieren mit den Kernpunkten: Rückgabe 2 (Kartennummern 350 und 402); Aufrufe für die nicht leeren Teilbäume mit den Wurzeln 518, 274, 402, 350, 461 (nicht: 730, 133, 615, 911, 688); Begründung: im BST sind alle Nummern im linken Teilbaum kleiner als x; ist x < von, sind auch sie < von und liegen nicht im Bereich – analog für x > bis; liegt x im Bereich, wird x gezählt und beide Teilbäume werden untersucht; da jede Nummer nur in einem Teilbaum steht, wird jede passende Nummer genau einmal gezählt. *Typische Punktverluste: P7, P10*		2	2	
n) Fehler in entferneVorgaenge finden und korrigieren mit den Kernpunkten: Fehler 1 (Zeile 3): <= statt < – im letzten Durchlauf getItem(getLength()) → Laufzeitfehler (Indexfehler); Fehler 2 (Zeile 5): == vergleicht Referenzen statt Inhalte, Kennzeichen werden ggf. nicht erkannt – equals verwenden; Fehler 3 (Zeilen 6/7): nach delete(i) rückt das nächste Element auf Index i, wird durch i++ übersprungen, z. B. zwei aufeinanderfolgende Vorgänge desselben Fahrzeugs – Korrektur: i-- nach delete, rückwärts laufen oder solange-Schleife mit i++ nur ohne Löschen. *Typische Punktverluste: P1, P5, P6*	1	3	1	
Summe	18	32	10	**/ 60**

Die vom Prüfling gewählten Lösungsansätze und -wege müssen nicht mit denen in den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (Ergänzende Hinweise 2025) entsprechend zu bewerten.

Notenschlüssel (60 BE)

Notenpunkte	15	14	13	12	11	10	09	08
ab Prozent	95	90	85	80	75	70	65	60
BE	57–60	54–56	51–53	48–50	45–47	42–44	39–41	36–38

Notenpunkte	07	06	05	04	03	02	01	00
ab Prozent	55	50	45	40	33	27	20	0
BE	33–35	30–32	27–29	24–26	20–23	17–19	12–16	0–11

Prozentgrenzen nach der in Niedersachsen üblichen Zuordnung (wie im Zentralabitur); BE-Grenzen = Prozentsatz · 60, aufgerundet. **Bitte an die Vorgabe der Schule anpassen** (z. B. Umgang mit halben BE, Fachkonferenzbeschluss).

Übersicht: Anforderungsbereiche und Kompetenzen

Teil	Kompetenz / Thema	I	II	III	BE	Zeit ca.
a)	Klassen (erläutern)	3	1		4	10 min
b)	Klassendiagramm, Vererbung	1	2		3	8 min
c)	dynamische Reihung (implementieren)	2	4		6	15 min
d)	Schlange, Struktur unverändert (implementieren)	2	4	1	7	18 min
e)	2D-Reihung (Struktogramm entwickeln)	1	4	1	6	15 min
f)	Trace (Tracetabelle)	2	3		5	13 min
g)	Sortierverfahren analysieren		2	2	4	10 min
h)	Effizienz: Zählen, Verdopplung	1	2	1	4	10 min
i)	Rekursionstabelle	2	2		4	10 min
j)	Rekursion erläutern	1	2		3	8 min
k)	Rekursion begründen		1	2	3	8 min
l)	binärer Suchbaum	2			2	5 min
m)	BinTree/BST: rekursive Operation analysieren		2	2	4	10 min
n)	Fehler finden (Java)	1	3	1	5	13 min
Σ		18	32	10	60	150 min + 30 min Einlesen/Kontrolle

AFB-Verteilung: I 18 BE (30,0 %) · II 32 BE (53,3 %) · III 10 BE (16,7 %) – wie im Zentralabitur Block 1 (18 / 32 / 10). Alle Zahlen, Tabellen und Musterlösungen wurden per Programm erzeugt bzw. geprüft (Python-Modell, javac 21 mit Nachbau der Bibliothek „Niedersachsen“).

Korrekturhinweise

- **Folgefehler** werden nicht erneut abgezogen: Wer in l) einen falschen Baum zeichnet und m) darauf richtig bearbeitet, erhält in m) die volle Punktzahl; Gleiches gilt für Folgefehler in den Tabellen f) und i) sowie für h) auf der Grundlage einer eigenen, schlüssigen Analyse aus g).
- **Alternative Lösungen** sind gleichwertig, wenn sie die Anforderungen erfüllen. Für c), d), e) und n) sind geprüfte Alternativen im Anhang abgedruckt. Bei Zweifeln die Handschrift unverändert in den Papier-Compiler (AQ_W6_Klausur_Papier-Compiler.txt) abtippen – ein roter Test ist ein Hinweis, die Bewertung erfolgt aber nach den Kernpunkten.
- **Syntax:** Kleine Schreibfehler ohne fachliche Bedeutung (fehlendes Semikolon, Groß-/Kleinschreibung) führen nicht zu Abzug. Fehlende Casts (P4) werden je Teilaufgabe höchstens mit 1 BE abgezogen. Sprachunabhängige Formulierungen in Implementierungen sind nicht zulässig; Java-Syntax im Struktogramm (P9) führt zu Abzug beim Notationspunkt.
- **Laufzeitfehler (P1):** Nicht ausgeschlossene Laufzeitfehler (Entnahme aus leerer Schlange, Index außerhalb, Zugriff auf null) führen zum Verlust des betreffenden Kernpunkts (EH §4).
- **Zeichenketten (P5):** Vergleich mit == statt equals ist fachlich falsch, auch wenn er in der Online-IDE funktioniert.

- **Operatoren (P10):** „erläutern“ verlangt Veranschaulichung mit Bezug zum Material, „begründen“ eine Rückführung auf Gesetzmäßigkeiten, „analysieren“ das Herausarbeiten der Funktionsweise; bloße Nennungen erreichen höchstens die AFB-I-Anteile.

Musterlösungen, Kernpunkte und typische Fehler

a) Aufbau der Klasse Ladesaeule erläutern 4 BE (I 3 · II 1 · III 0)

Musterlösung

Eine Klasse beschreibt den Bauplan für gleichartige Objekte. Die **Attribute** nummer, leistung und aktuell legen fest, welche Eigenschaften jedes Ladesäulen-Objekt besitzt; aktuell ist eine Referenz auf das Ladevorgang-Objekt, das gerade lädt (null, wenn die Säule frei ist).

Die **Konstruktoren** (mit c gekennzeichnet) sind besondere Operationen, mit denen Objekte erzeugt und ihre Attribute initialisiert werden. Die Klasse besitzt zwei Konstruktoren mit unterschiedlichen Parameterlisten (Überladen): Ladesaeule(nummer) erzeugt eine Säule mit der Standardleistung 11 kW, Ladesaeule(nummer, leistung) übernimmt zusätzlich die übergebene Leistung. In beiden Fällen ist die neue Säule frei.

Die **Operationen** beschreiben das Verhalten: getNummer, getLeistung und istFrei fragen den Zustand ab, starten und beenden verändern ihn (aktuell wird gesetzt bzw. auf null zurückgesetzt), preisProKWh berechnet einen Wert.

Sichtbarkeit: Attribute mit - sind privat und nur innerhalb der Klasse zugreifbar, Operationen mit + sind öffentlich. So ist der Zustand gekapselt: Andere Klassen (z. B. Ladepark) können leistung nur über getLeistung lesen; da es keinen Setter gibt, ist die Leistung nach der Erzeugung nicht mehr veränderbar.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Attribute als Eigenschaften mit Datentyp, aktuell als Referenz (null = frei)	I	1	☐
Konstruktoren als besondere Operationen zur Objekterzeugung; zwei Konstruktoren mit unterschiedlicher Parameterliste (Standardleistung 11 kW bzw. übergebene Leistung)	I	1	☐
Operationen: Abfrage-, Veränderungs- und Berechnungsoperationen mit Beispiel aus der Klasse	I	1	☐
Sichtbarkeit - / + erläutert, Datenkapselung am Beispiel (leistung nur über getLeistung lesbar, kein Setter → nach der Erzeugung unveränderlich)	II	1	☐

Alternativen / Teilpunkte: Andere passende Beispiele aus der Klasse sind gleichwertig. Eine bloße Aufzählung der Bestandteile ohne Bezug zur Klasse Ladesaeule erreicht höchstens 2 BE (Operator „erläutern“, P10).

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P10** Bestandteile nur genannt, nicht erläutert; kein Bezug auf die Klasse.
- **P10** Zwei Konstruktoren als „Fehler“ gedeutet statt als Überladen.
- **–** Sichtbarkeit verwechselt (- als öffentlich).

b) Klassendiagramm um Schnellladesaeule ergänzen 3 BE (I 1 · II 2 · III 0)

Musterlösung

Lösungsskizze (Notation nach EH §6; geerbte Elemente werden nicht wiederholt):

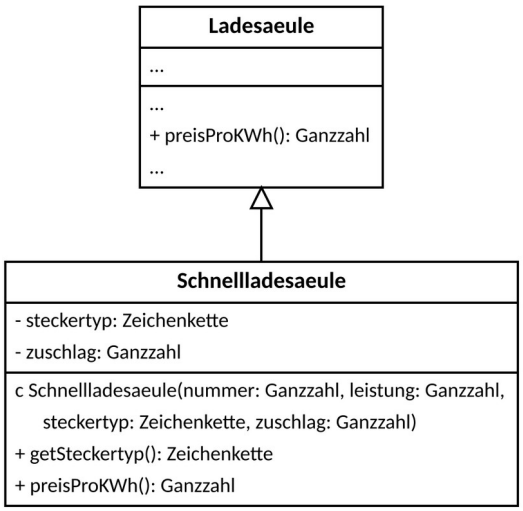


Abbildung EH.1: Ergänzung des Klassendiagramms

Die Java-Fassung steht im Papier-Compiler (Klasse Schnellladesaeule, Konstruktor mit super(nummer, leistung), preisProKWh() gibt super.preisProKWh() + zuschlag zurück).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Klasse Schnellladesaeule mit Vererbungspfeil (hohle Spitze) zur Oberklasse Ladesaeule	I	1	☐
Attribute steckertyp, zuschlag mit Sichtbarkeit und Typ; Konstruktor mit allen nötigen Parametern	II	1	☐
Überschreiben von preisProKWh(): Ganzzahl (Zuschlag), keine Wiederholung geerbter Elemente	II	1	☐

Alternativen / Teilpunkte: Ein fester Zuschlag als Konstante oder ohne Attribut direkt in preisProKWh ist gleichwertig, wenn er erkennbar ist. Ein Konstruktor ohne Parameter leistung (Schnellladesäule mit fester Leistung, z. B. 150 kW) ist bei Begründung gleichwertig. Assoziationslinie statt Vererbungspfeil: Punkt 1 nicht erreicht.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P3** Signatur von preisProKWh geändert (Parameter ergänzt) – dann wird nicht überschrieben.
- **–** Pfeil in falscher Richtung oder geschlossene Spitze.
- **–** Alle Attribute der Oberklasse erneut aufgeführt (kein Abzug, aber Zeitverlust).

c) besteFreieSaeule implementieren (dynamische Reihung) 6 BE (I 2 · II 4 · III 0)

Musterlösung

Java (Papier-Compiler: TeilC_BesteFreieSaeuleTest 6/6 grün):

```
// Teilaufgabe c) besteFreieSaeule(): Ladesaeule
public Ladesaeule besteFreieSaeule() {
    Ladesaeule beste = null;
    for (int i = 0; i < saeulen.getLength(); i++) {
        Ladesaeule s = (Ladesaeule) saeulen.getItem(i);
        if (s.istFrei()) {
            if (beste == null || s.getLeistung() > beste.getLeistung()) {
                beste = s;
            }
        }
    }
    return beste;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf mit Rückgabetyp Ladesaeule, Rückgabe am Ende	I	1	☐
Durchlauf aller Elemente ohne Indexfehler (0 … getLength() − 1 oder for-each)	I	1	☐
Cast (Ladesaeule) beim Auslesen	II	1	☐
nur freie Säulen berücksichtigt (istFrei())	II	1	☐
Maximum der Leistung, erstes Maximum bei Gleichstand (strenges >)	II	1	☐
null bei leerer Reihung bzw. ohne freie Säule; kein Zugriff auf null (beste == null zuerst prüfen)	II	1	☐

Alternativen / Teilpunkte: Gleichwertig (Papier-Compiler grün): for-each mit Index-Zähler, Hilfsvariable für den Index des bisher besten Elements, zweistufiges Vorgehen (erste freie Säule suchen, dann vergleichen). Fehlt der Cast, wird höchstens 1 BE abgezogen. Eine Abfrage saeulen.isEmpty() ist nicht nötig, wenn die Schleife bei leerer Reihung nicht durchlaufen wird.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P1** beste.getLeistung() aufgerufen, solange beste noch null ist (NullPointerException).
- **P1/P6** Schleife bis i <= getLength() (IndexOutOfBounds).
- **P4** Cast (Ladesaeule) vergessen.
- **–** >= statt > (liefert bei Gleichstand die letzte Säule).
- **P1** Startwert beste = saeulen.getItem(0) – liefert ggf. eine belegte Säule bzw. Laufzeitfehler bei leerer Reihung.

d) wartezeit implementieren (Schlange unverändert) 7 BE (I 2 · II 4 · III 1)

Musterlösung

Java (Papier-Compiler: TeilD_WartezeitTest 6/6 grün):

```
// Teilaufgabe d) wartezeit(kennzeichen: Zeichenkette): Ganzzahl
public int wartezeit(String kennzeichen) {
    Queue hilf = new Queue();
    int summe = 0;
    boolean gefunden = false;
    while (!warteschlange.isEmpty()) {
        Ladevorgang v = (Ladevorgang) warteschlange.dequeue();
        if (v.getKennzeichen().equals(kennzeichen)) {
            gefunden = true;
        }
        if (!gefunden) {
            summe = summe + v.getDauer();
        }
        hilf.enqueue(v);
    }
    while (!hilf.isEmpty()) {
        warteschlange.enqueue(hilf.dequeue());
    }
    if (gefunden) {
        return summe;
    }
    return -1;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf (Parameter Zeichenkette, Rückgabe Ganzzahl), Hilfsschlange bzw. Zähler	I	1	☐
Durchlauf mit isEmpty() vor jedem dequeue(), Cast (Ladevorgang)	I	1	☐
Vergleich der Kennzeichen mit equals	II	1	☐
Summe nur der Ladedauern vor dem gesuchten Ladevorgang	II	1	☐
Rückgabe −1, wenn das Kennzeichen nicht vorkommt	II	1	☐
Schlange vollständig in alter Reihenfolge wiederhergestellt	II	1	☐
Suche und Wiederherstellung korrekt verzahnt (keine Rückgabe vor der Wiederherstellung, kein vorzeitiger Abbruch)	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (Papier-Compiler grün): einmaliges Rotieren der Schlange, nachdem ihre Länge bestimmt wurde; Ergebnis beim Treffer merken und weiter umfüllen. Ein return direkt beim Treffer ist nur richtig, wenn vorher die gesamte Schlange wiederhergestellt wurde. Die Online-IDE vergleicht Zeichenketten auch mit == inhaltlich – im Papier-Compiler fällt == daher nicht auf, ist aber nach EH falsch (P5): Punkt „equals“ nicht erreicht.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P2** Hilfsschlange nicht zurückgeführt – warteschlange ist danach leer.
- **P2** Beim Treffer abgebrochen und nur die Hilfsschlange hinten angehängt – Reihenfolge verändert.
- **P5** Kennzeichen mit == verglichen.
- **P1** head() bzw. dequeue() ohne Leer-Abfrage, z. B. in der Schleifenbedingung.
- **–** Eigene Ladedauer mitgezählt bzw. 0 statt −1 bei fehlendem Kennzeichen.

e) Struktogramm spitzenstunde entwickeln (2D-Reihung) 6 BE (I 1 · II 4 · III 1)

Musterlösung

Musterlösung (zeilengetreu in Java übertragen und mit dem Papier-Compiler geprüft: TeilE_SpitzenstundeTest 5/5 grün; Testdaten aus Abbildung 3 ergänzt um die übrigen Stunden → 17):

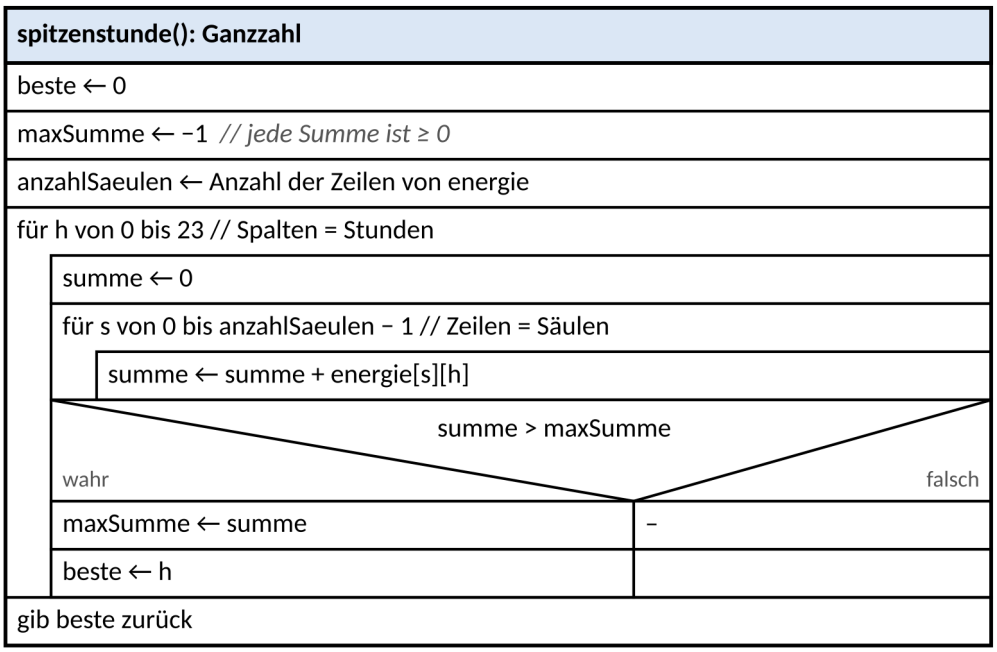


Abbildung EH.2: Struktogramm der Operation spitzenstunde

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Kopf mit Signatur, Rückgabe der Stunde mit „gib … zurück“	I	1	☐
äußere Schleife über alle Stunden 0 bis 23 (Spalten)	II	1	☐
innere Schleife über alle Säulen, Summe je Stunde zurückgesetzt, Zugriff energie[s][h] (Zeile = Säule, Spalte = Stunde)	II	1	☐
Maximum und zugehörige Stunde gemerkt, früheste Stunde bei Gleichstand (strenges >)	II	1	☐
Notation nach EH §5 (←, Zählschleifen, Verzweigung, keine Java-Syntax)	II	1	☐
spaltenweiser Durchlauf als Umkehrung der üblichen Zeilen-Reihenfolge sachgerecht gelöst, Startwert des Maximums passend (−1 oder Summe der Stunde 0)	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: Startwert maxSumme ← Summe der Stunde 0 und Schleife ab Stunde 1; Zählschleife „für h von 0 bis (Anzahl der Spalten) − 1“; Berechnung aller 24 Summen in einer Hilfsreihung und anschließende Maximumsuche. Java-Code statt Struktogramm: höchstens 4 von 6 BE (P9). Vertauschte Indizes energie[h][s]: Punkt 3 nicht erreicht (Laufzeitfehler bzw. falsches Ergebnis).

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P6** Indizes vertauscht (energie[h][s]) oder Zeilen statt Spalten summiert (liefert die stärkste Säule).
- **P6** Schleife nur bis 22 bzw. bis zur Anzahl der Säulen.
- **–** summe nicht je Stunde auf 0 gesetzt.
- **–** ≥ statt > (späteste statt früheste Stunde).
- **P9** Java-Syntax (for (int h = 0; …), ==, return) im Struktogramm, fehlender Kopf.

f) Ablauf von sortiereProtokoll dokumentieren (Tracetabelle) 5 BE (I 2 · II 3 · III 0)

Musterlösung

Vollständige Tabelle (mit dem vorgegebenen Algorithmus in Java und unabhängig in Python erzeugt und verglichen):

Durchlauf	pos (Beginn)	Vergleich	[0]	[1]	[2]	[3]	[4]	pos (Ende)
Start	1	–	18	42	25	42	9	
1	1	wahr	42	18	25	42	9	0
2	0	–	42	18	25	42	9	1
3	1	falsch	42	18	25	42	9	2
4	2	wahr	42	25	18	42	9	1
5	1	falsch	42	25	18	42	9	2
6	2	falsch	42	25	18	42	9	3
7	3	wahr	42	25	42	18	9	2
8	2	wahr	42	42	25	18	9	1
9	1	falsch	42	42	25	18	9	2
10	2	falsch	42	42	25	18	9	3
11	3	falsch	42	42	25	18	9	4
12	4	falsch	42	42	25	18	9	5

Grün: Vertauschung (insgesamt 4). Wert von pos nach Ende der Schleife: 5. Endbelegung: 5 Ladevorgänge mit 42, 42, 25, 18, 9 kWh (HB-AQ 300, WST-M 5, DEL-LP 42, OL-EV 7, BRA-K 19).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
erste Durchläufe korrekt: Tausch bei pos = 1, Sonderfall pos = 0	I	1	☐
eine Zeile je Schleifendurchlauf mit pos und Vergleichsergebnis (Konvention der Tabelle)	I	1	☐
Belegungen nach den Vertauschungen korrekt (18 wandert nach hinten, zweite 42 wandert zwei Plätze nach vorn)	II	2	☐
Endbelegung 42, 42, 25, 18, 9, Ende bei pos = 5	II	1	☐

Alternativen / Teilpunkte: Folgefehler: Ein Fehler in einer Zeile wird nur einmal abgezogen, wenn danach richtig weitergearbeitet wird. Die zusätzliche Spalte „pos am Ende“ oder das Weglassen unveränderter Belegungen (mit „s. o.“) ist zulässig. Es sind genau 12 Durchläufe; das entspricht (n − 1) + 2 · Anzahl der Vertauschungen = 4 + 2 · 4.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P8** Nur Zeilen mit Vertauschung notiert.
- **P8** Bei gleichen Werten (42, 42) getauscht – der Vergleich ist strikt (<).
- **–** Nach einem Tausch mit pos + 1 weitergemacht (wie Bubblesort).

g) Sortierverfahren analysieren 4 BE (I 0 · II 2 · III 2)

Musterlösung

Funktionsweise: pos bezeichnet das Element, das gerade mit seinem linken Nachbarn verglichen wird. Ist der linke Nachbar kleiner, werden beide vertauscht und pos geht eine Stelle zurück – das Element „wandert“ so lange nach links, bis links von ihm ein mindestens gleich großer Wert steht. Danach läuft pos wieder nach rechts bis zum nächsten Element, das noch nicht an seiner Stelle ist. Das entspricht der

Einfügesortierung, bei der das Einfügen durch wiederholte Nachbarvertauschungen erfolgt (Verfahren „Gnome Sort“).

Abfrage pos = 0: Ist ein Element bis an den Anfang gewandert, hat es keinen linken Nachbarn. Ohne die Abfrage würde getItem(pos − 1) = getItem(−1) einen Laufzeitfehler auslösen. In diesem Fall wird pos einfach erhöht.

Begründung der Sortierung: Zu Beginn jedes Durchlaufs ist der Bereich protokoll[0] … protokoll[pos − 1] absteigend sortiert (am Anfang besteht er nur aus Element 0). Wird pos erhöht, dann nur, wenn links ≥ rechts gilt – der sortierte Bereich wächst um ein Element. Bei einer Vertauschung wird pos verringert; der Bereich links von pos bleibt sortiert, weil nur das neue Element in ihn hinein wandert. Jede Vertauschung beseitigt genau eine Fehlstellung (Paar in falscher Reihenfolge); da es nur endlich viele gibt, erreicht pos schließlich die Länge der Reihung. Dann ist der Bereich 0 … Länge − 1, also die ganze Reihung, absteigend sortiert. Da bei gleichen Energiewerten nicht getauscht wird, ist das Verfahren stabil.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Funktionsweise: Element wandert durch Nachbartausch nach links bis zu seiner Stelle, danach geht pos wieder vorwärts (Einfügesortierung durch Vertauschen)	II	1	☐
Bedeutung von pos = 0: kein linker Nachbar, Zugriff getItem(−1) wird vermieden (Laufzeitfehler), pos wird erhöht	II	1	☐
Begründung über die Invariante: Bereich 0 … pos − 1 ist stets absteigend sortiert; beim Tausch bleibt er sortiert, pos + 1 nur bei richtiger Reihenfolge	III	1	☐
Terminierung/Schluss: jede Vertauschung beseitigt eine Fehlstellung, also endet die Schleife mit pos = Länge – dann ist die ganze Reihung sortiert	III	1	☐

Alternativen / Teilpunkte: Eine Begründung am Beispiel aus f) mit erkennbarer Verallgemeinerung erreicht die Punkte 1 und 2 sowie höchstens einen der beiden III-Punkte. Die Benennung des Verfahrens ist nicht verlangt.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P10** Ablauf nur nacherzählt („erst wird getauscht, dann …“), ohne zu begründen, warum am Ende sortiert ist.
- **P1** pos = 0 als „Optimierung“ gedeutet statt als Schutz vor dem Zugriff auf Index −1.
- **–** Verwechslung mit Bubblesort (dort laufen ganze Durchgänge über die Reihung).

h) Vertauschungen bestimmen, Verdopplung abschätzen 4 BE (I 1 · II 2 · III 1)

Musterlösung

Absteigend sortiert: Jeder Vergleich ergibt falsch, pos läuft ohne Rückschritt von 1 bis n: **0 Vertauschungen** (n − 1 Schleifendurchläufe).

Streng aufsteigend sortiert: Das Element mit Index k ist größer als alle k Elemente links von ihm. Es wird k-mal mit dem linken Nachbarn vertauscht, bis es ganz vorn steht. Insgesamt: 1 + 2 + … + (n − 1) = **n(n − 1)/2 Vertauschungen** (das ist der ungünstigste Fall: jedes Paar steht in falscher Reihenfolge).

Verdopplung: Für 2n Ladevorgänge ergeben sich 2n(2n − 1)/2 = n(2n − 1) Vertauschungen, also etwa das **Vierfache** von n(n − 1)/2 (für große n gilt (2n − 1)/(n − 1) ≈ 2, zusammen mit dem Faktor 2 also ≈ 4). Die Laufzeit wächst quadratisch und vervierfacht sich ungefähr.

Zahlenbeispiele (per Programm geprüft): n = 10: 45 · n = 20: 190 · n = 100: 4.950 · n = 200: 19.900 · n = 1000: 499.500 · n = 2000: 1.999.000.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
bereits absteigend sortiert: 0 Vertauschungen	I	1	☐
aufsteigend: Element mit Index k wird k-mal nach vorn getauscht	II	1	☐
Summe 1 + 2 + … + (n − 1) = n(n − 1)/2	II	1	☐

Kernpunkt	AFB	BE	erreicht
Abschätzung bei Verdopplung: etwa Faktor 4 (quadratisch), mit Begründung	III	1	☐

Alternativen / Teilpunkte: Eine Abschätzung über die Anzahl der Schleifendurchläufe ((n − 1) + 2 · Vertauschungen, also $n^2 - 1$ im ungünstigsten Fall) ist gleichwertig. Für die aufsteigende Reihung genügt eine nachvollziehbare Herleitung an einem Beispiel mit allgemeiner Formel.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P10** Nur „doppelt so lange“ ohne Begründung oder ohne Bezug auf die Formel.
- **–** Vergleiche statt Vertauschungen gezählt und mit Vertauschungen vermischt.
- **–** n^2 statt n(n − 1)/2 angegeben, ohne Herleitung (Teilpunkt).

i) Rekursionstabelle zu ueberlast 4 BE (I 2 · II 2 · III 0)

Musterlösung

Vollständige Rekursionstabelle (per Programm in Java und Python erzeugt und verglichen):

Rekursionstiefe	h	puffer beim Aufruf	bedarf[h]	puffer nach der Berechnung	Rückgabe
0 (Erstaufruf)	0	60	20	120 (140 → begrenzt)	4
1	1	120	130	90	4
2	2	90	160	30	4
3	3	30	90	40	4
4	4	40	150	-10 < 0: Überlast	4

Rückgabe des Erstaufrufs: 4 (Überlast in Stunde 4). Die Stunden 5 bis 7 werden nicht mehr betrachtet.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Rekursionstiefe und h (0 bis 4, Erstaufruf = Tiefe 0)	I	1	☐
Abbruch in Stunde 4, keine weiteren Aufrufe	I	1	☐
Pufferwerte je Aufruf einschließlich der Begrenzung auf die Kapazität 120	II	1	☐
Rückgabe 4 in jeder Zeile (Rückgabe wird durchgereicht)	II	1	☐

Alternativen / Teilpunkte: Folgefehler bei den Pufferwerten werden nur einmal abgezogen. Wird in der Spalte „puffer nach der Berechnung“ in Zeile 0 der unbegrenzte Wert 140 statt 120 eingetragen und danach mit 120 weitergerechnet, ist dies nicht zu beanstanden.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P7** Rekursionstiefe beim Erstaufruf mit 1 begonnen.
- **P7/P8** Rückgabe nur in der letzten Zeile eingetragen.
- **–** Kapazitätsgrenze nicht beachtet (140 statt 120) – dann Überlast erst später.

j) Prinzip der Rekursion erläutern 3 BE (I 1 · II 2 · III 0)

Musterlösung

Rekursion bedeutet, dass eine Operation sich selbst mit einem kleineren Teilproblem aufruft, bis ein direkt lösbarer Fall (Basisfall) erreicht ist. ueberlast bearbeitet nur die Stunde h: Sie berechnet den neuen Pufferstand und überlässt die restlichen Stunden h + 1, h + 2, … dem Selbstaufruf ueberlast(bedarf, h + 1, puffer). Der aktuelle Zustand des Pufferspeichers wird dabei als Parameter weitergegeben.

Es gibt zwei Abbruchbedingungen: Ist h gleich der Länge der Prognose, sind alle Stunden ohne Überlast verarbeitet und −1 wird zurückgegeben; ist der Pufferstand negativ, wird die aktuelle Stunde h zurückgegeben. Da h bei jedem Selbstaufruf um 1 wächst, wird spätestens bei h = Länge ein Basisfall

erreicht. Das Ergebnis des Selbstaufrufs wird unverändert an den Aufrufer zurückgegeben (vgl. Spalte Rückgabe in i)).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Selbstaufruf mit verkleinertem Problem (h + 1, verbleibende Stunden) und aktuellem Pufferstand als Parameter	I	1	☐
zwei Abbruchbedingungen (h = Länge → −1; puffer < 0 → h) als Basisfälle	II	1	☐
Rückgabe des Selbstaufrufs wird weitergereicht; Terminierung, weil h wächst und die Länge erreicht	II	1	☐

Alternativen / Teilpunkte: Eine allgemeine Erläuterung der Rekursion ohne Bezug auf ueberlast erreicht höchstens 1 BE (P10).

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P10** Nur „die Methode ruft sich selbst auf“ ohne Basisfall und Verkleinerung.
- **P7** Nur eine Abbruchbedingung genannt.

k) Rekursionstiefe begründen 3 BE (I 0 · II 1 · III 2)

Musterlösung

Beim Erstaufruf ist h = 0 und die Rekursionstiefe 0. Jeder Selbstaufruf erfolgt mit h + 1 und erhöht die Rekursionstiefe um 1. Also hat jeder Aufruf die Rekursionstiefe h. Die Länge von bedarf sei n.

Gibt die Operation −1 zurück, stammt dieser Wert aus dem Basisfall h = n (der andere Basisfall liefert h ≥ 0, und Selbstaufrufe geben ihr Ergebnis nur weiter). Der letzte Aufruf hat also h = n und damit die Rekursionstiefe n.

Umgekehrt: Hat der letzte Aufruf die Rekursionstiefe n, dann ist dort h = n. Dieser Aufruf kam nur zustande, weil in den Stunden 0 bis n − 1 jeweils puffer ≥ 0 galt, also keine Überlast auftrat; er gibt −1 zurück, und dieser Wert wird durchgereicht. Tritt dagegen in einer Stunde h < n eine Überlast auf, endet die Rekursion dort mit der Tiefe h < n (Beispiel i): Tiefe 4 < 8).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Tiefe eines Aufrufs = sein Parameter h (beide beginnen bei 0 und wachsen je Selbstaufruf um 1)	II	1	☐
Rückgabe −1 ⇒ letzter Aufruf im Basisfall h = Länge, also Tiefe = Länge	III	1	☐
Umkehrung: Tiefe = Länge ⇒ keine Überlast in den Stunden 0 … n − 1 ⇒ Rückgabe −1 (bzw. Überlast ⇒ Tiefe < Länge)	III	1	☐

Alternativen / Teilpunkte: Eine Begründung, die nur eine Richtung zeigt, erreicht höchstens 2 BE. Ein Hinweis auf das Beispiel aus i) ersetzt nicht die allgemeine Begründung.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P7** Erstaufruf mit Tiefe 1 gezählt – dann scheint die Tiefe n + 1 zu sein.
- **P10** Nur am Beispiel argumentiert („bei i) war es 4“).

l) Binären Suchbaum zeichnen 2 BE (I 2 · II 0 · III 0)

Musterlösung

Binärer Suchbaum nach dem Einfügen (Höhe 4):

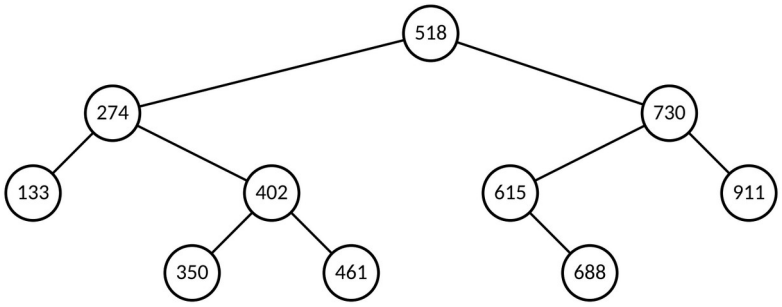


Abbildung EH.3: binärer Suchbaum karten

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Wurzel und Ebene 1 (274, 730) sowie 133, 402, 615, 911 richtig eingeordnet	I	1	☐
350, 461 unter 402 und 688 als rechtes Kind von 615	I	1	☐

Alternativen / Teilpunkte: Folgefehler in m) werden nicht erneut abgezogen.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- – 688 als linkes Kind von 730 eingefügt (Vergleich mit 615 vergessen).
- – Einfügen in der Reihenfolge der Größe statt der vorgegebenen Reihenfolge.

m) anzahlImBereich analysieren 4 BE (I 0 · II 2 · III 2)

Musterlösung

Ablauf: 518 > 450 → nur linker Teilbaum; 274 < 300 → nur rechter Teilbaum; 402 liegt im Bereich → zählt, beide Teilbäume; 350 liegt im Bereich → zählt (Teilbäume leer); 461 > 450 → linker Teilbaum (leer).
Rückgabe: 2. Aufgerufen wird für die nicht leeren Teilbäume mit den Wurzeln 518, 274, 402, 350, 461; die Teilbäume mit 730, 133, 615, 911, 688 werden nicht untersucht (insgesamt 8 Aufrufe einschließlich der leeren Teilbäume).

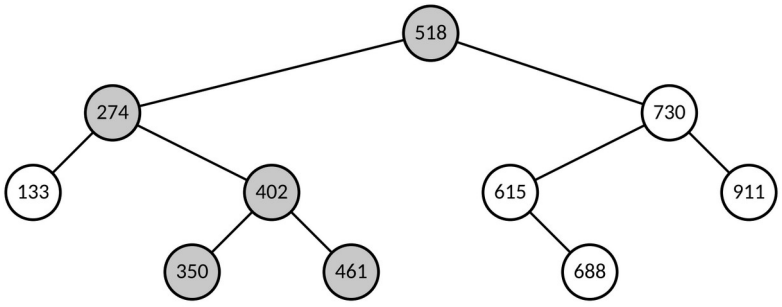


Abbildung EH.4: grau – Wurzeln der nicht leeren Teilbäume, für die anzahlImBereich aufgerufen wird

Begründung: In einem binären Suchbaum sind alle Nummern im linken Teilbaum eines Knotens kleiner als die Nummer x des Knotens. Gilt x < von, so sind alle Nummern im linken Teilbaum ebenfalls kleiner als von und liegen nicht im Bereich – sie brauchen nicht gezählt zu werden; x selbst liegt auch nicht im Bereich. Nur im rechten Teilbaum können passende Nummern stehen. Für x > bis gilt das Entsprechende mit dem rechten Teilbaum. Liegt x im Bereich, wird x mit 1 gezählt und beide Teilbäume werden durchsucht. Da jede Nummer in genau einem Teilbaum steht, wird jede Nummer im Bereich genau einmal gezählt. Dadurch werden oft große Teile des Baums übersprungen (hier 5 von 10 Knoten).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Rückgabe 2	II	1	☐
besuchte nicht leere Teilbäume: Wurzeln 518, 274, 402, 350, 461	II	1	☐
Begründung über die BST-Eigenschaft: linker Teilbaum enthält nur Werte < x < von (analog rechts bei x > bis)	III	1	☐
Vollständigkeit: im Bereich wird x gezählt und beide Teilbäume untersucht, jede Nummer genau einmal	III	1	☐

Alternativen / Teilpunkte: Die Angabe der Aufrufe als Liste mit Parametern (z. B. anzahlImBereich(Teilbaum mit Wurzel 274, 300, 450)) ist gleichwertig; die zusätzliche Angabe der Aufrufe für leere Teilbäume ist kein Fehler.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P10** Nur „weil der Baum sortiert ist“ ohne Bezug auf die Lage der Werte im Teilbaum.
- **–** Bei x < von den linken Teilbaum gewählt (Richtung vertauscht).
- **P7** Knoten 461 nicht als besucht angegeben (der Aufruf erfolgt, gezählt wird er nicht).

n) Fehler in entferneVorgaenge finden und korrigieren 5 BE (I 1 · II 3 · III 1)

Musterlösung

Fehler und Auswirkungen: **Zeile 3** <= → im letzten Durchlauf gilt i = getLength(), getItem(i) löst einen Laufzeitfehler aus (P1/P6). **Zeile 5** == vergleicht bei Zeichenketten die Referenzen, nicht die Inhalte – ein übergebenes, gleich lautendes Kennzeichen wird ggf. nicht erkannt (P5). **Zeilen 6/7** nach delete(i) rückt das folgende Element auf Index i und wird durch i++ übersprungen: Folgen zwei Vorgänge desselben Fahrzeugs direkt aufeinander, bleibt einer stehen – bei OL-EV 7, HB-AQ 300, HB-AQ 300, DEL-LP 42, HB-AQ 300 und dem Kennzeichen HB-AQ 300 werden nur 2 statt 3 Vorgänge entfernt, es bleibt OL-EV 7, HB-AQ 300, DEL-LP 42 (P6). Korrigierte Fassung (Papier-Compiler: TeilN_EntferneVorgaengeTest 5/5 grün):

```
// Teilaufgabe n) entferneVorgaenge – korrigiert (Aenderungen markiert)
public int entferneVorgaenge(String kennzeichen) {
    int anzahl = 0;
    for (int i = 0; i < protokoll.getLength(); i++) {  // Fehler 1: < statt <=
        Ladevorgang v = (Ladevorgang) protokoll.getItem(i);
        if (v.getKennzeichen().equals(kennzeichen)) {  // Fehler 2: equals statt ==
            protokoll.delete(i);
            i--;  // Fehler 3: Index korrigieren
            anzahl++;
        }
    }
    return anzahl;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
drei Fehler angegeben (Zeile 3, Zeile 5, Zeilen 6/7)	I	1	☐
Auswirkung Fehler 1: Zugriff auf Index getLength() → Laufzeitfehler	II	1	☐
Auswirkung Fehler 2: Referenz- statt Inhaltsvergleich → Kennzeichen evtl. nicht erkannt	II	1	☐
Auswirkung Fehler 3 mit Beispiel: aufeinanderfolgende Treffer – eines wird übersprungen (z. B. OL-EV 7, HB-AQ 300, HB-AQ 300, DEL-LP 42, HB-AQ 300 → nur 2 statt 3 entfernt)	II	1	☐

Kernpunkt	AFB	BE	erreicht
Korrekturen, insbesondere sachgerechte Korrektur von Fehler 3 (i-- / rückwärts / solange-Schleife)	III	1	☐

Alternativen / Teilpunkte: Gleichwertige Korrekturen von Fehler 3 (Papier-Compiler grün): rückwärts von getLength() − 1 bis 0 laufen; solange-Schleife, in der i nur erhöht wird, wenn nicht gelöscht wurde. Die Korrektur „i < getLength() − 1“ für Fehler 1 ist falsch (letzter Ladevorgang wird nicht geprüft). Wird ein Fehler gefunden, der keiner ist (z. B. fehlende Leer-Abfrage – die Schleife wird bei leerer Reihung nicht durchlaufen), wird nicht abgezogen, wenn die drei Fehler genannt sind.

Typische Fehler (Bezug zu den Top 10 Punktverlusten)

- **P6** Fehler 3 übersehen (häufigster Fehler).
- **P5** == als richtig angesehen, weil es in der Online-IDE funktioniert.
- **P1/P6** Fehler 1 durch getLength() − 1 „korrigiert“.

Anhang: Papier-Compiler und geprüfte Alternativen

Die Datei **AQ_W6_Klausur_Papier-Compiler.txt** (nur Lehrkraft) enthält die Kontextklassen, die vorgegebenen Operationen aus den Abbildungen 4, 7 und 10, Testklassen im JUnit-Dialekt der Online-IDE und die Musterlösung als markierten Kommentarblock. Bibliothek „Niedersachsen“ im Workspace aktivieren; die Handschrift unverändert in die passende Methode abtippen und im Reiter „Testrunner“ ausführen.

Testklasse	Tests	Musterlösung	Gerüst
TeilC_BesteFreieSaeuleTest	6	6/6 grün	0/6 grün
TeilD_WartezeitTest	6	6/6 grün	0/6 grün
TeilE_SpitzenstundeTest	5	5/5 grün	0/5 grün
TeilN_EntferneVorgaengeTest	5	5/5 grün	0/5 grün
VorgabenTest	3	3/3 grün	3/3 grün

Vor Einsatz prüfen

In der Online-IDE vergleicht == Zeichenketten inhaltlich – Varianten mit == bleiben dort grün (bei d) und n) von Hand prüfen, P5). Casts (Ladesaeule)/(Ladevorgang)/(Integer) und den Testrunner vor dem ersten Einsatz einmal mit der Musterlösung durchlaufen lassen. Geprüft wurde mit javac (Java 21) und Nachbauten von Queue, DynArray und BinTree mit denselben Exceptions.

Typische Fehlvarianten (Mutanten) – müssen und werden rot

Teil	P	Variante	Tests grün
c)	P4	Cast (Ladesaeule) vergessen	Kompilierfehler
c)	P6/P1	Schleife bis i <= getLength()	0/6
c)	P1	beste == null nicht abgefragt (NullPointer)	0/6 – in der Online-IDE ggf. grün (== vergleicht dort inhaltlich): von Hand prüfen
c)	–	istFrei() nicht abgefragt	3/6
c)	–	Vergleich mit >= (bei Gleichstand die letzte)	4/6
c)	P1	Start mit beste = Element 0 (belegt/leer nicht beachtet)	3/6
c)	–	Rückgabe bei der ersten freien Säule	4/6
c)	P3	Rückgabe fehlt am Ende	Kompilierfehler
d)	P2	Schlange nicht wiederhergestellt (Hilfsschlange nicht zurückgeführt)	1/6
d)	P2	Abbruch beim Treffer, nur Hilfsschlange hinten angehängt	3/6
d)	P5	Kennzeichen mit == verglichen	1/6 – in der Online-IDE ggf. grün (== vergleicht dort inhaltlich): von Hand prüfen
d)	P6	eigene Ladedauer mitgezählt	1/6
d)	–	0 statt −1, wenn nicht enthalten	4/6
d)	P1	head() ohne Leer-Abfrage in der Schleifenbedingung	0/6
d)	P4	Cast (Ladevorgang) vergessen	Kompilierfehler
e)	P6	Indizes vertauscht: energie[h][s]	0/5
e)	–	Vergleich mit >= (späteste statt früheste Stunde)	3/5
e)	P6	Schleife nur bis Stunde 22 (h < 23)	4/5

Teil	P	Variante	Tests grün
e)	–	summe nicht je Stunde zurückgesetzt	2/5
e)	P3	maxSumme statt Stunde zurückgegeben	0/5
n)	P1/P6/P5	Originalfassung aus Abbildung 11 (alle drei Fehler)	0/5
n)	P5/P6	nur Fehler 1 (<=) korrigiert	0/5
n)	P6	Fehler 1 und 2 korrigiert, Fehler 3 nicht (Element nach dem gelöschten übersprungen)	3/5
n)	P5	Fehler 1 und 3 korrigiert, == bleibt	0/5 – in der Online-IDE ggf. grün (== vergleicht dort inhaltlich): von Hand prüfen
n)	P6	Korrektur von Fehler 1 als i < getLength() – 1 (letztes Element fehlt)	0/5

Gleichwertige Alternativen (alle Tests der Teilaufgabe grün)

c) Alternative 1: for-each und Vergleich über den Index [6/6 grün]

```
public Ladesaeule besteFreieSaeule() {
    int besterIndex = -1;
    int maxLeistung = -1;
    int i = 0;
    for (Object o : saeulen) {
        Ladesaeule s = (Ladesaeule) o;
        if (s.istFrei() && s.getLeistung() > maxLeistung) {
            maxLeistung = s.getLeistung();
            besterIndex = i;
        }
        i++;
    }
    if (besterIndex == -1) {
        return null;
    }
    return (Ladesaeule) saeulen.getItem(besterIndex);
}
```

c) Alternative 2: Leer-Abfrage vorab, zwei Schritte (erste freie, dann besser) [6/6 grün]

```
public Ladesaeule besteFreieSaeule() {
    if (saeulen.isEmpty()) {
        return null;
    }
    Ladesaeule beste = null;
    int i = 0;
    while (i < saeulen.getLength() && beste == null) {
        Ladesaeule s = (Ladesaeule) saeulen.getItem(i);
        if (s.istFrei()) {
            beste = s;
        }
        i++;
    }
    while (i < saeulen.getLength()) {
        Ladesaeule s = (Ladesaeule) saeulen.getItem(i);
        if (s.istFrei() && s.getLeistung() > beste.getLeistung()) {
            beste = s;
        }
        i++;
    }
    return beste;
}
```

d) Alternative 1: Länge bestimmen und Schlange einmal rotieren (ohne Hilfsschlange) [6/6 grün]

```
public int wartezeit(String kennzeichen) {
    Queue zaehl = new Queue();
    int n = 0;
    while (!warteschlange.isEmpty()) {
        zaehl.enqueue(warteschlange.dequeue());
        n++;
    }
    while (!zaehl.isEmpty()) {
        warteschlange.enqueue(zaehl.dequeue());
    }
    int summe = 0;
    int ergebnis = -1;
    for (int i = 0; i < n; i++) {
        Ladevorgang v = (Ladevorgang) warteschlange.dequeue();
        if (ergebnis == -1 && v.getKennzeichen().equals(kennzeichen)) {
            ergebnis = summe;
        }
        summe = summe + v.getDauer();
        warteschlange.enqueue(v);
    }
    return ergebnis;
}
```

d) Alternative 2: Rückgabe sofort beim Treffer, Rest vorher umfüllen [6/6 grün]

```
public int wartezeit(String kennzeichen) {
    Queue hilf = new Queue();
    int summe = 0;
    int ergebnis = -1;
    while (!warteschlange.isEmpty()) {
        Ladevorgang v = (Ladevorgang) warteschlange.dequeue();
        hilf.enqueue(v);
        if (ergebnis == -1) {
            if (v.getKennzeichen().equals(kennzeichen)) {
                ergebnis = summe;
            } else {
                summe = summe + v.getDauer();
            }
        }
    }
    while (!hilf.isEmpty()) {
        warteschlange.enqueue(hilf.dequeue());
    }
    return ergebnis;
}
```

e) Alternative: feste Stundenzahl 24, Start mit der Summe von Stunde 0 [5/5 grün]

```
public int spitzenstunde() {
    int maxSumme = 0;
    for (int s = 0; s < energie.length; s++) {
        maxSumme = maxSumme + energie[s][0];
    }
    int beste = 0;
    for (int h = 1; h <= 23; h++) {
        int summe = 0;
        for (int s = 0; s < energie.length; s++) {
            summe = summe + energie[s][h];
        }
        if (summe > maxSumme) {
            maxSumme = summe;
            beste = h;
        }
    }
    return beste;
}
```

n) Alternative 1: Rückwärts durchlaufen [5/5 grün]

```
public int entferneVorgaenge(String kennzeichen) {
    int anzahl = 0;
    for (int i = protokoll.getLength() - 1; i >= 0; i--) {
        Ladevorgang v = (Ladevorgang) protokoll.getItem(i);
        if (v.getKennzeichen().equals(kennzeichen)) {
            protokoll.delete(i);
            anzahl++;
        }
    }
    return anzahl;
}
```

n) Alternative 2: solange-Schleife, i nur ohne Löschen erhöhen [5/5 grün]

```
public int entferneVorgaenge(String kennzeichen) {
    int anzahl = 0;
    int i = 0;
    while (i < protokoll.getLength()) {
        Ladevorgang v = (Ladevorgang) protokoll.getItem(i);
        if (v.getKennzeichen().equals(kennzeichen)) {
            protokoll.delete(i);
            anzahl++;
        } else {
            i++;
        }
    }
    return anzahl;
}
```

Musterlösung zu e) in Java (zeilengetreue Übertragung von Abbildung EH.2):

```
// Teilaufgabe e) spitzenstunde(): Ganzzahl  (Struktogramm EH Abbildung E.2)
public int spitzenstunde() {
    int beste = 0;
    int maxSumme = -1;
    for (int h = 0; h < energie[0].length; h++) {
        int summe = 0;
        for (int s = 0; s < energie.length; s++) {
            summe = summe + energie[s][h];
        }
        if (summe > maxSumme) {
            maxSumme = summe;
            beste = h;
        }
    }
    return beste;
}
```